

the value. That way, if we need to eliminate a given account, no other account is affected. A dictionary of accounts would look like this:

```
{0 : <object for account 0>, 1 : <object for account 1>, ... }
```

We can then easily get the associated `Account` object and call a method like this:

```
oAccount = accountsDict[accountNumber]
oAccount.someMethodCall()
```

Alternatively, we could use the `accountNumber` directly to make a call to a method of an individual `Account`:

```
accountsDict[accountNumber].someMethodCall()
```

[Listing 4-5](#) shows the test code using a dictionary of `Account` objects. Again, while we're making many changes to our test code, we're not changing a single line in the `Account` class. In our test code, rather than using hardcoded account numbers, we add a counter, `nextAccountNumber`, that we'll increment after creating a new `Account`.

File:
BankOOP3_DictionaryOfAccountObjects/Main_Bank_Version3.py

```
# Test program using accounts
# Version 3, using a dictionary of accounts

# Bring in all the code from the Account class file
from Account import *

accountsDict = {} ❶
nextAccountNumber = 0 ❷

# Create two accounts:
oAccount = Account('Joe', 100, 'JoesPassword')
joesAccountNumber = nextAccountNumber
accountsDict[joesAccountNumber] = oAccount ❸
print('Account number for Joe is:', joesAccountNumber)
```